```
name: daily-trivia-short

on:
  schedule:
    - cron: "17 21 * * *"   # 毎日 日本時間 06:17
  workflow_dispatch:         # 手動実行ボタン

permissions:
  contents: write

concurrency:
  group: daily-trivia-short
  cancel-in-progress: false

jobs:
  make:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: 部品のインストール
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y -qq ffmpeg fonts-noto-cjk
          pip install -q edge-tts requests google-api-python-client google-auth

      - name: 動画を作って投稿
        env:
          GEMINI_API_KEY: ${{ secrets.GEMINI_API_KEY }}
          YT_CLIENT_ID: ${{ secrets.YT_CLIENT_ID }}
          YT_CLIENT_SECRET: ${{ secrets.YT_CLIENT_SECRET }}
          YT_REFRESH_TOKEN: ${{ secrets.YT_REFRESH_TOKEN }}
          YT_PRIVACY: ${{ vars.YT_PRIVACY }}
        run: |
          cat > make_short.py <<'PYEOF'
          #!/usr/bin/env python3
          """日本語雑学ショート動画を1本作り、(設定があれば)YouTubeに投稿する。"""
          import asyncio
          import glob
```

```
import json
import os
import pathlib
import random
import re
import subprocess
import sys
import time
import urllib.parse

import requests

OUT = pathlib.Path("out")
WORK = pathlib.Path("work")
VOICE = os.environ.get("TTS_VOICE", "ja-JP-NanamiNeural")
MODELS = [
    m.strip()
    for m in os.environ.get(
        "GEMINI_MODELS", "gemini-2.5-flash,gemini-2.5-flash-lite,gemini
    ).split(",")
    if m.strip()
]
UA = {"User-Agent": "trivia-shorts/1.0 (personal hobby project)"}
CATEGORIES = [
    "動物", "宇宙", "食べ物と飲み物", "人体のふしぎ", "日本の歴史", "日本の文化と
    "日本語と言葉", "身近な科学", "世界の地理", "乗り物と交通", "スポーツの豆知識
    "天気と自然現象", "植物", "お金と経済の仕組み", "建物と街",
]
PALETTES = [
    ("0x1b2a49", "0x6a3093"), ("0x0f2027", "0x2c5364"), ("0x42275a", "0
    ("0x134e5e", "0x71b280"), ("0x3a1c71", "0xd76d77"), ("0x1f1c2c", "0
]

_last_call = [0.0]


def log(msg):
    print(msg, flush=True)


def run(cmd):
    r = subprocess.run(cmd, capture_output=True, text=True)
    if r.returncode != 0:
        raise RuntimeError(
            "コマンド失敗: " + " ".join(map(str, cmd))[:300] + "\n" + r.s
        )
    return r.stdout
```

```
# ------------------------------------------------------------------ Gemi
def gemini(prompt):
    key = os.environ.get("GEMINI_API_KEY", "").strip()
    if not key:
        raise RuntimeError("GEMINI_API_KEY が設定されていません")
    last = "モデル未試行"
    for model in MODELS:
        url = f"https://generativelanguage.googleapis.com/v1beta/models
        for attempt in range(3):
            wait = 4.5 - (time.time() - _last_call[0])
            if wait > 0:
                time.sleep(wait)
            _last_call[0] = time.time()
            try:
                r = requests.post(
                    url,
                    headers={"x-goog-api-key": key},
                    json={
                        "contents": [{"parts": [{"text": prompt}]}],
                        "generationConfig": {
                            "responseMimeType": "application/json",
                            "temperature": 0.8,
                        },
                    },
                    timeout=120,
                )
            except requests.RequestException as e:
                last = f"{model}: 通信エラー {e}"
                log("  " + last)
                time.sleep(5)
                continue
            if r.status_code == 200:
                try:
                    text = r.json()["candidates"][0]["content"]["parts"
                    text = re.sub(r"^```(?:json)?\s*|\s*```$", "", text
                    return json.loads(text)
                except Exception as e:  # noqa: BLE001
                    last = f"{model}: 応答の解析に失敗 {e}"
                    log("  " + last)
                    continue
            last = f"{model}: HTTP {r.status_code} {r.text[:300]}"
            log("  Geminiエラー: " + last)
            if r.status_code in (429, 500, 503):
                time.sleep(15 * (attempt + 1))
                continue
```

```
            break  # 404/400など：次のモデルへ
    raise RuntimeError("Geminiの呼び出しに失敗しました：" + last)


# ---------------------------------------------------------------- ネタ
def pick_candidates(used):
    cat = random.choice(CATEGORIES)
    log(f"  カテゴリ：{cat}")
    prompt = (
        "日本語の雑学YouTubeショート向けに、意外性があり話題にしやすいテーマを8個考
        f"カテゴリ：{cat}\n"
        "条件：日本語版Wikipediaに独立した記事がある項目名(article)にすること。"
        "特定の人物の私生活・政治・宗教・事件・災害・医療効果の断定は避けること。\n
        f"次の項目は除外：{', '.join(used[-150:]) or 'なし'}\n"
        'JSON形式：{"candidates":[{"article":"記事名","hook":"どんな意外な点
    )
    data = gemini(prompt)
    return [c for c in data.get("candidates", []) if isinstance(c, dict


def wiki_extract(article):
    r = requests.get(
        "https://ja.wikipedia.org/w/api.php",
        params={
            "action": "query", "prop": "extracts", "explaintext": 1, "r
            "titles": article, "format": "json", "formatversion": 2,
        },
        headers=UA,
        timeout=30,
    )
    r.raise_for_status()
    pages = r.json().get("query", {}).get("pages", [])
    if not pages or pages[0].get("missing"):
        return None
    p = pages[0]
    text = (p.get("extract") or "").strip()
    if len(text) < 500 or "曖昧さ回避" in text[:300]:
        return None
    title = p["title"]
    return title, text[:5000], "https://ja.wikipedia.org/wiki/" + urlli


def write_script(title, src, hook):
    prompt = (
        "あなたは日本語の雑学ショート動画の台本作家です。\n"
        "以下の【資料】だけを根拠に、約30秒（全体で150〜190文字）のオリジナル台本を
        "ルール:\n"
```

```
        "- 資料に書かれていない事実・数字・年号は絶対に使わない\n"
        "- 資料の文章をそのまま写さず、自分の言葉で言い換える\n"
        "- 1文目は『えっ？』と思わせる問いかけか意外な事実にする\n"
        "- 1文は40文字以内、全体で5〜7文、話し言葉（です・ます調）\n"
        "- 最後の1文は短い一言のまとめにする\n"
        "- 『諸説あります』が必要になる内容は避ける\n\n"
        f"【ネタの狙い】{hook}\n【記事名】{title}\n【資料】\n{src}\n\n"
        'JSON形式: {"video_title":"20文字以内のタイトル","sentences":["文1"
    )
    return gemini(prompt)


def script_ok(sc):
    try:
        s = sc["sentences"]
        if not (4 <= len(s) <= 9) or not all(isinstance(x, str) and x.s
            return False
        total = sum(len(x) for x in s)
        if not (110 <= total <= 230) or max(len(x) for x in s) > 55:
            return False
        t = sc["video_title"]
        return isinstance(t, str) and 0 < len(t) <= 28
    except Exception:  # noqa: BLE001
        return False


def verify(src, sc):
    prompt = (
        "あなたは厳格なファクトチェッカーです。【台本】の全ての主張が【資料】の内容
        "資料に無い事実・数字・年号・断定的な言い切りが1つでもあれば ok を false
        f"【資料】\n{src}\n\n【台本】\n" + "\n".join(sc["sentences"]) + "\
        'JSON形式: {"ok":true または false,"problems":["問題点"]}'
    )
    return gemini(prompt)


# ---------------------------------------------------------------- 音声
async def synth_all(sentences):
    import edge_tts

    for i, text in enumerate(sentences):
        path = WORK / f"a{i}.mp3"
        for attempt in range(4):
            try:
                await edge_tts.Communicate(text, VOICE, rate="+0%").sav
                if path.exists() and path.stat().st_size > 1000:
                    break
```

```
            except Exception as e:  # noqa: BLE001
                log(f"  音声生成リトライ {attempt + 1}: {e}")
                await asyncio.sleep(3 * (attempt + 1))
        else:
            raise RuntimeError(f"edge-ttsで音声を作れませんでした（{i + 1}文


def duration(path):
    out = run([
        "ffprobe", "-v", "error", "-show_entries", "format=duration",
        "-of", "default=nw=1:nk=1", str(path),
    ])
    return float(out.strip())


def build_narration(n):
    """文ごとの音声をつなぎ、(開始秒, 終了秒)のリストと全体の長さを返す。"""
    wavs, times, t = [], [], 0.0
    for i in range(n):
        wav = WORK / f"w{i}.wav"
        af = "apad=pad_dur=0.35"
        if i == 0:
            af = "adelay=500,apad=pad_dur=0.35"
        run(["ffmpeg", "-y", "-i", str(WORK / f"a{i}.mp3"), "-ar", "441
             "-af", af, str(wav)])
        d = duration(wav)
        times.append((t, t + d))
        t += d
        wavs.append(wav)
    lst = WORK / "list.txt"
    lst.write_text("".join(f"file '{w.resolve()}'\n" for w in wavs), en
    run(["ffmpeg", "-y", "-f", "concat", "-safe", "0", "-i", str(lst),
         str(WORK / "narration.wav")])
    return times, t


# ---------------------------------------------------------------- 動画
def wrap(text, n=11):
    lines, cur = [], ""
    for ch in text:
        cur += ch
        if (len(cur) >= n and ch in "、。！？, ") or len(cur) >= n + 3:
            lines.append(cur)
            cur = ""
    if cur:
        lines.append(cur)
    if len(lines) > 1 and len(lines[-1]) <= 2 and len(lines[-2]) + len(
```

```
        lines[-2] += lines.pop()
    return lines


def find_font():
    for pat in ["/usr/share/fonts/opentype/noto/NotoSansCJK-Bold.ttc",
                "/usr/share/fonts/opentype/noto/NotoSansCJK-Black.ttc",
                "/usr/share/fonts/**/NotoSansCJK*Bold*", "/usr/share/fo
                "/usr/share/fonts/**/*CJK*", "/usr/share/fonts/**/*Goth
        hits = sorted(glob.glob(pat, recursive=True))
        if hits:
            return hits[0]
    raise RuntimeError("日本語フォントが見つかりません")


def render(video_title, sentences, times, total, out_path):
    font = find_font()
    log(f"  フォント: {font}")
    parts = []
    idx = 0

    def add(line, y, size, color, border, enable):
        nonlocal idx
        tf = WORK / f"t{idx}.txt"
        idx += 1
        tf.write_text(line, encoding="utf-8")
        parts.append(
            f"drawtext=fontfile={font}:textfile={tf}:expansion=none:fon
            f":fontcolor={color}:borderw={border}:bordercolor=black@0.9
            f":x=(w-text_w)/2:y={y}" + (f":enable='{enable}'" if enable
        )

    for k, line in enumerate(wrap(video_title, 9)):
        add(line, 230 + k * 120, 88, "0xFFD54A", 8, None)
    for (a, b), s in zip(times, sentences):
        lines = wrap(s)
        top = 1000 - len(lines) * 100 // 2
        for k, line in enumerate(lines):
            add(line, top + k * 100, 68, "white", 7, f"between(t,{a:.2f
    graph = WORK / "graph.txt"
    graph.write_text("[0:v]" + ",".join(parts) + "[v]", encoding="utf-8
    c0, c1 = random.choice(PALETTES)
    bg = (f"gradients=s=1080x1920:r=30:d={total + 0.5:.2f}:c0={c0}:c1={
          ":x0=0:y0=0:x1=1080:y1=1920:speed=0.02")
    run(["ffmpeg", "-y", "-f", "lavfi", "-i", bg, "-i", str(WORK / "nar
         "-filter_complex_script", str(graph), "-map", "[v]", "-map", "
         "-c:v", "libx264", "-preset", "veryfast", "-crf", "24", "-pix_
```

```
            "-c:a", "aac", "-b:a", "160k", "-t", f"{total:.2f}", "-movflag
            str(out_path)])


def check_video(path):
    out = run(["ffprobe", "-v", "error", "-show_entries",
               "stream=codec_type,width,height", "-show_entries", "form
               "-of", "json", str(path)])
    info = json.loads(out)
    kinds = {s["codec_type"] for s in info["streams"]}
    v = next(s for s in info["streams"] if s["codec_type"] == "video")
    if (v["width"], v["height"]) != (1080, 1920) or "audio" not in kind
        raise RuntimeError(f"動画の形式が想定と違います: {info}")
    return float(info["format"]["duration"])


# ---------------------------------------------------------------- YouT
def upload_youtube(video, title, desc, tags):
    need = ["YT_CLIENT_ID", "YT_CLIENT_SECRET", "YT_REFRESH_TOKEN"]
    if not all(os.environ.get(k, "").strip() for k in need):
        log("  YouTubeの設定がまだ無いため、投稿はスキップします")
        return None
    from google.oauth2.credentials import Credentials
    from googleapiclient.discovery import build
    from googleapiclient.errors import HttpError
    from googleapiclient.http import MediaFileUpload

    creds = Credentials(
        None,
        refresh_token=os.environ["YT_REFRESH_TOKEN"].strip(),
        token_uri="https://oauth2.googleapis.com/token",
        client_id=os.environ["YT_CLIENT_ID"].strip(),
        client_secret=os.environ["YT_CLIENT_SECRET"].strip(),
        scopes=["https://www.googleapis.com/auth/youtube.upload"],
    )
    yt = build("youtube", "v3", credentials=creds, cache_discovery=Fals
    body = {
        "snippet": {"title": title[:95], "description": desc, "tags": t
                    "categoryId": "27"},
        "status": {"privacyStatus": os.environ.get("YT_PRIVACY", "priva
                   "selfDeclaredMadeForKids": False},
    }
    req = yt.videos().insert(
        part="snippet,status", body=body,
        media_body=MediaFileUpload(str(video), mimetype="video/mp4", re
    )
    try:
```

```
            resp = None
            while resp is None:
                _, resp = req.next_chunk()
        except HttpError as e:
            raise RuntimeError(f"YouTube投稿エラー: {e.status_code} {e.conter
        return resp.get("id"), resp.get("status", {}).get("privacyStatus")


    # ---------------------------------------------------------------- メイン
    def main():
        OUT.mkdir(exist_ok=True)
        WORK.mkdir(exist_ok=True)
        used_file = pathlib.Path("used_topics.txt")
        used = [x.strip() for x in used_file.read_text(encoding="utf-8").sp
            if used_file.exists() else []

        log("[1/6] ネタ選び")
        chosen = None
        for cand in pick_candidates(used)[:8]:
            art = str(cand.get("article", "")).strip()
            if not art or art in used:
                continue
            got = wiki_extract(art)
            if not got:
                log(f"  スキップ(記事が不適切): {art}")
                continue
            title, text, url = got
            if title in used:
                continue
            log(f"[2/6] 台本作成と事実確認: {title}")
            for attempt in range(2):
                sc = write_script(title, text, str(cand.get("hook", "")))
                if not script_ok(sc):
                    log("  台本の形式/長さが基準外。作り直します")
                    continue
                v = verify(text, sc)
                if v.get("ok") is True:
                    chosen = (title, url, sc)
                    break
                log("  事実確認で不合格: " + json.dumps(v.get("problems", []),
            if chosen:
                break
        if not chosen:
            raise RuntimeError("事実確認に通るネタが見つかりませんでした（再実行して
        title, url, sc = chosen
        sentences = [s.strip() for s in sc["sentences"]]
        log("  台本:\n    " + "\n    ".join(sentences))
```

```
        log("[3/6] ナレーション生成")
        asyncio.run(synth_all(sentences))
        times, total = build_narration(len(sentences))
        log(f"  音声の長さ: {total:.1f}秒")
        if total > 58:
            raise RuntimeError(f"音声が長すぎます({total:.1f}秒)。再実行してくだ

        log("[4/6] 動画生成")
        video = OUT / "short.mp4"
        render(sc["video_title"], sentences, times, total, video)
        log("[5/6] 動画チェック")
        log(f"  OK 1080x1920 / {check_video(video):.1f}秒")

        desc = (
            "\n".join(sentences)
            + f"\n\n出典: {url}（Wikipedia）\n※台本はAIが作成し、資料との照合で事実
        )
        tags = [str(t) for t in sc.get("tags", [])] + ["雑学", "Shorts"]
        yt_title = f"{sc['video_title']} #Shorts"
        (OUT / "info.json").write_text(
            json.dumps({"title": yt_title, "description": desc, "tags": tag
                        ensure_ascii=False, indent=2), encoding="utf-8")
        with used_file.open("a", encoding="utf-8") as f:
            f.write(title + "\n")

        log("[6/6] YouTube投稿")
        res = upload_youtube(video, yt_title, desc, tags)
        if res:
            log(f"  投稿完了: https://youtu.be/{res[0]} (公開設定: {res[1]})")
        log("完了")


    if __name__ == "__main__":
        try:
            main()
        except Exception as e:  # noqa: BLE001
            log(f"エラー: {e}")
            sys.exit(1)

    PYEOF
    python make_short.py

- name: 動画を保存（7日間）
  if: always()
  uses: actions/upload-artifact@v4
  with:
```

```
      name: short-video
      path: out/
      if-no-files-found: ignore
      retention-days: 7

  - name: 使ったネタを記録
    if: success()
    run: |
      git config user.name "github-actions"
      git config user.email "github-actions@users.noreply.github.com"
      git add used_topics.txt
      git diff --cached --quiet || (git commit -m "使用済みネタを記録" && git pu
```